v1.0 · April 2026

Pier

The *complete* user guide.

Chat-driven file and database editing for legacy WordPress, Joomla, Drupal, Magento and custom PHP sites — over FTP/FTPS, with safe proposals and automatic backups.

pier.abn.company

A Brand New Company · Amsterdam / Sisaket

CONTENTS

What's in this guide

01 What Pier is (and is not) 03

02 Installing & activating your license 04

03 Adding a connection 06

04 How Pier explores your server 08

05 Talking to the AI 09

06 The full tool list 10

07 Safe edits: proposals, diffs, undo 13

08 Auto-audits (SEO, security, performance…) 15

09 Databases (MySQL) 16

10 Backups & snapshots 19

11 Settings & preferences 20

12 Keyboard shortcuts 21

13 Troubleshooting 22

14 Licenses & offline use 24

15 Privacy & security 25

16 FAQ 26

CHAPTER 01

What Pier is (and is not)

Pier is a native macOS application that connects to your web hosting and lets you edit files and databases by **talking to an AI**. Instead of opening an FTP client, finding the right file, editing it in a text editor and re-uploading, you type what you want — "add a cookie banner to the homepage" or "find every page that still links to the old domain" — and Pier does the reading, the diffing, and (with your permission) the writing.

The three ideas Pier is built on

1. **Proposal-based edits.** Pier never writes to your server without your approval. Every change is shown as a before/after diff that you Accept or Reject.
2. **Local-first.** Your FTP credentials, license and chat history live on *your* Mac, encrypted with AES-256-GCM. Pier does not upload your site to any cloud.
3. **Automatic safety net.** Every edit is backed up before it's applied. There's a 10-second one-click undo on every change and you can snapshot an entire folder as a zip at any time.

What Pier can do today (v1.0)

- Connect to FTP and FTPS (explicit TLS) servers.
- Detect domains and CMSes (WordPress, Joomla, Drupal, Laravel, Magento, PrestaShop) automatically on first connect.
- Discover database credentials from config files — `wp-config.php`, `configuration.php`, `.env`, `app/etc/env.php`, and more.
- Read, edit, rename, delete, upload and download files via chat or UI.
- Query MySQL databases (read-only by default, writes behind an extra confirmation step).
- Run audit agents — SEO, security, performance, accessibility, broken links, meta descriptions, viewport, analytics, external links, and a full-site audit.
- Keep versioned per-file backups and full-folder zip snapshots.

What Pier does *not* do yet

BE HONEST WITH YOUR USERS — AND YOURSELF

- **No SSH-based SFTP.** Pier speaks FTP and FTPS (explicit TLS). True SFTP over SSH is on the v1.1 roadmap.
- **No PostgreSQL / SQLite.** The database bridge is MySQL-only. Other engines are not detected or queried.
- **No schema changes.** `CREATE`, `ALTER` and `DROP` are blocked at the bridge. Pier does not rewrite schemas — only rows.
- **No trial period.** Pier 1.0 requires a purchased license to use the AI chat and file editing. You can still browse files once activated; there is no free tier.

- **macOS only.** No Windows or Linux builds. Minimum macOS 12 (Monterey); universal binary for Apple Silicon and Intel.

Installing & activating your license

1. Download

Go to `pier.abn.company/download` and grab `Pier-1.0.0.dmg`. The DMG is universal (Apple Silicon + Intel), code-signed with an Apple Developer ID, and notarized by Apple.

If you want to verify the download, the SHA-256 is published alongside the DMG at `/download/Pier-1.0.0.sha256`. Run:

```
shasum -a 256 ~/Downloads/Pier-1.0.0.dmg
# compare with the value on the site
```

2. Install

1. Double-click the DMG to mount it.
2. Drag **Pier.app** into **Applications**.
3. Eject the DMG.
4. Open Pier from Launchpad. macOS may show a "Pier cannot be opened because Apple cannot check it for malicious software" prompt the very first time — this is Gatekeeper being cautious on a first-run of a newly notarized app. Right-click Pier.app → **Open** → **Open**.

3. Activate

After purchase you receive an email from `info@abn.company` with your license key (starts with `pier_`) and a PDF receipt. On first launch Pier shows an activation screen.

1. Paste your license key. Pier strips whitespace, so wrapped pastes from mail clients are fine.
2. Click **Activate**. Pier verifies the signature locally (~0.3 ms), then calls the license server to claim one of your two seats.
3. On success, the app unlocks and you land on the connections screen.

> ABOUT THE 2 SEATS
>
> One license covers two Macs — a desktop and a laptop, a work and a home machine, whatever you want. Each seat is bound to the machine's hardware UUID, so moving the app between disks on the same Mac doesn't re-claim a seat, but cloning onto a different Mac does.

4. Deactivating a seat

To free a seat for a different Mac, open Pier → **Settings → License → Deactivate this device**. This contacts the license server and releases the slot. Deactivation is instant and reversible — your key keeps working, you just need to re-activate whichever Mac you want to use next.

If you've lost access to the Mac a seat is bound to (sold it, wiped it, dead hardware), email `info@abn.company` with your order ID and we'll release the seat on the server side.

5. Re-validation and the 7-day grace window

Pier re-validates your license with the server every 7 days in the background. If the server is unreachable (travel, flaky Wi-Fi, a Vercel outage), Pier keeps working for another 7 days from the last successful validation. After that you'll see a gentle warning in Settings; after 60 days offline the AI chat is gated until you come back online. File browsing and connection management never lock.

Adding a connection

A **connection** is a saved set of credentials for one FTP or FTPS server. Most users have one per hosting account; agencies often have one per client site.

The connection form

Field	What to enter
Name	A label only you see. "Client X — staging" is more useful than the hostname.
Host	Hostname or IP. No `ftp://` prefix.
Port	Default `21`. Some providers use `990` for implicit FTPS (not supported — use FTPS on 21 or 990 with explicit TLS).
Protocol	**FTP** (plaintext, legacy) or **FTPS (TLS)** (recommended whenever the host supports it).
Username	Your FTP user. Often the same as the cPanel/DirectAdmin user.
Password	Stored encrypted (AES-256-GCM) in `~/Library/Application Support/Pier/connections.json`. Never sent anywhere except to your FTP server.
Working folder	Optional. A path like `/domains/example.com/public_html`. All file operations — AI-driven or manual — are bounded to this folder. Prevents the AI from wandering into `/mail` or `/etc`.
Read-only	When checked, every write operation (upload, edit apply, rename, delete) returns an error. Useful for production connections you don't want to modify by accident.

Test the connection

Before saving, click **Test**. Pier opens a control socket, authenticates, and lists the first directory. On success you see a green tick and a file count; on failure you see one of the following classified errors:

Code	Meaning
`AUTH`	Username or password rejected. Also caught: account locked by the hosting provider.
`BANNED`	Your IP is temporarily blocked by fail2ban (often after repeated wrong passwords). Wait 15–60 minutes and try again.
`TIMEOUT`	The server didn't respond within 30 s. Usually a firewall or wrong port.
`NOT_FOUND`	Host name didn't resolve. Check for typos.
`TRANSIENT`	Hosted server briefly unavailable. Retry in a minute.

Quality-of-life notes

- **One active control socket per connection.** Writes serialize; reads can parallelize if you opt in (see Chapter 11 — `PIER_READ_POOL_SIZE`). The safe default is serial because parallel reads can trip shared-hosting fail2ban.
- **Idle timeout: 10 minutes.** Pier auto-disconnects after 10 minutes of inactivity. Reconnects happen transparently on the next action.

- **Circuit breaker.** After 3 control-socket failures in 60 s, the breaker opens for 180 s to protect you from getting IP-banned. During that window the connection reports `unavailable`; you can force-reset from the sidebar.

> FTPS, NOT SFTP
>
> Pier's "secure" protocol is **FTPS (explicit TLS)** — the FTP protocol wrapped in TLS, running on port 21 with an `AUTH TLS` upgrade. This is *not* the same as SSH-based SFTP (port 22). Most shared hosts offer FTPS. If yours only speaks SFTP, see the v1.1 roadmap on pier.abn.company/blog.

How Pier explores your server

Pier User Guide v1.0

The first time you connect to a server, Pier silently runs a **hosting structure scan**. You don't click anything — the scan starts the moment the connection test succeeds, and its result feeds every subsequent chat.

What the scan does

1. **Looks for a DirectAdmin-style layout** — a top-level `/domains/` folder with one subfolder per domain. For each, it finds the web root (tries `public_html`, `httpdocs`, `htdocs`, `www`, `web`, `html`, `public`, `webroot`) and any subdomains in `[domain]/subdomains`.
2. **Falls back to cPanel-style layout** — a single web root at the server root.
3. **Falls back to flat layout** — a site whose files are served directly from `/`.

CMS detection

Inside each web root Pier checks for CMS signatures. A CMS is reported as detected when at least two of its signature files are present.

CMS	Signatures Pier looks for
WordPress	`wp-config.php`, `wp-content`, `wp-includes`, `wp-admin`
Joomla	`configuration.php`, `components`, `modules`, `plugins`, `templates`
Drupal	`sites`, `core`, `modules`, `themes`, `profiles`
Laravel	`artisan`, `app`, `bootstrap`, `config`, `resources`, `routes`
Magento	`app/Mage.php`, `skin`, `media`, `var`
PrestaShop	`classes`, `controllers`, `modules`, `themes`, `config/settings.inc.php`

Database discovery

In parallel, Pier parses known config-file locations to discover database credentials. For WordPress that's `wp-config.php`; for Joomla `configuration.php`; for Laravel `.env`; for Magento `app/etc/env.php`; and so on — including CodeIgniter, OpenCart, and a generic PHP/env fallback. If the direct scan finds nothing, Pier opens entry files (`index.php`, `wp-load.php`, `bootstrap.php`) and follows their `require` / `include` chains to find config files buried deeper.

> NO DATABASE CONNECTION HAPPENS DURING SCAN
>
> Discovery is pure file parsing. Pier never opens a SQL socket to your database until you explicitly install the bridge and ask it a query.

The domain picker

If the scan finds more than one domain, Pier shows a **domain picker** card at the top of your first chat. Click a domain to pin it as the working folder for that session — every subsequent `read_file`, `propose_edit`, etc. is bounded to that domain.

Talking to the AI

The main interface is a chat panel. You type. Pier reads your message, figures out what tools it needs to call, calls them, and replies. Under the hood it's Claude Opus 4.7 by default, or Claude Sonnet 4.6 if you want faster/cheaper runs (see below).

How to phrase requests

Pier responds well to concrete, action-oriented prompts. A few patterns that work:

- *"Show me the homepage hero section in the active theme."* — Pier will walk the theme folder, find the active theme, identify the hero template, and read it.
- *"Every page that still links to example-old.com, replace with example-new.com."* — Pier runs content search, opens each match, proposes individual replace operations you can accept one by one or in bulk.
- *"Add a GDPR cookie banner at the bottom of footer.php, with an accept button."* — Pier reads `footer.php`, drafts the new HTML, proposes an edit.
- *"Find the WooCommerce product with SKU ABC-123 and mark it out of stock."* — Pier proposes a `propose_enable_db_writes` first, then an `UPDATE` once you've approved write mode.

Language

Pier's intent parser recognizes both **English** and **Dutch**. You can switch mid-conversation ("pas de homepagina aan" and "I want to edit the contact form" both work). The model replies in whichever language the last user message was in.

Switching the model

Each connection can use its own model. Open a connection → **Settings → AI Model** and pick:

- **Claude Opus 4.7** (default) — best reasoning, highest quality diffs, slower and more expensive to run if you use a BYO key.
- **Claude Sonnet 4.6** — ~3× faster, roughly 90% of the quality. Recommended for repetitive editing runs like bulk replace.

Bring-your-own API key

Pier ships with a shared inference quota, but for heavy users or agencies we recommend bringing your own Anthropic key. Connection → **Settings → API Key** → paste. The key is encrypted AES-256-GCM alongside your FTP credentials and never leaves the device.

> PRIVACY NOTE
>
> When Pier calls the AI, it sends the current user message, the relevant file contents it needs for the task, and a short summary of the conversation so far. It **does not** send your FTP password,

database passwords, or your license key. Secrets found inside files (API keys, database passwords in `wp-config.php`, Stripe/AWS/GitHub tokens) are redacted before the file is shown to the AI.

The full tool list

Everything the AI can do is one of these tools. When Pier calls a tool, the call appears in the chat as a small grey line — `read_file("/wp-config.php")`, `list_files("/wp-content/themes")`. Reading the tool trace is the fastest way to understand what Pier is doing and why.

File exploration

list_files

`{ path }`

List a directory. Returns folders and files with names, sizes, dates, and permissions. Cached for 60 s to keep follow-up questions snappy; any write operation invalidates the cache.

read_file

`{ path, force? }`

Read a file's content. Size is truncated to 40 KB in the AI's view, but the full file is held in memory for edits. Sensitive paths (`wp-config.php`, `.env`, `.htpasswd`, `*.key`) are redacted before reaching the model.

search_files

`{ path, pattern }`

Find files by name (regex). Returns up to 50 matches.

search_content

`{ path, query, extensions? }`

Full-text search inside files. Bounded at 200 files, depth 5, 40 s wall-clock. Returns up to 100 hits with line numbers.

get_page_structure

`{ path }`

Follow PHP `include` / `require` chains from a starting file to map how a page is composed (header → sidebar → footer). Great for "where do I actually edit the logo?" questions.

File edits (every one is a proposal)

propose_replace

`{ path, find, replace, reason }`

Small targeted edit: finds a unique string in the file and replaces it. If `find` matches zero or more than one occurrence, the tool errors so the AI refines its context.

propose_edit

`{ path, new_content, reason }`

Full-file rewrite. Shows a line-level diff in the chat card.

propose_create

`{ path, content, reason, based_on? }`

Create a new file. Pier pre-checks the path doesn't exist so you never overwrite silently.

propose_delete

`{ path, reason }`

Delete a file. Auto-flagged as high-risk for `.htaccess`, `wp-config.php`, `web.config`, `configuration.php`, `composer.json`, `package.json`.

create_folder

`{ path }`

Make a new directory (runs immediately — low-risk).

rename_file

`{ from, to }`

Rename or move a file (runs immediately).

set_working_folder

`{ path }`

Pin a folder as the session's working root. Persisted per chat.

Tool list (continued)

Snapshots & backups

`create_snapshot`

`{ name }`

Zip the entire working folder to `~/Library/Application Support/Pier/backups/[conn]/_snapshots/` with SHA-256 checksum. Used before risky refactors.

`list_backups`

`{ path }`

Show per-file version history for one path, newest first. Every edit leaves a `.bak` alongside the local cached file.

Databases

`list_databases`

`{ }`

Return every database discovered during the scan. Includes CMS, host, database name, username, port, table prefix. Credentials stay encrypted on disk — only metadata is returned.

`list_db_tables`

`{ db_index? }`

Show tables in one of the discovered databases. Returns name, row count, engine, size. Requires the bridge to be installed (Chapter 09).

`query_database`

`{ sql, db_index? }`

Run a read-only query. Only `SELECT`, `SHOW`, `DESCRIBE`, `EXPLAIN`, `WITH` are allowed — the bridge rejects everything else. Row cap: 500 (truncation flagged). Query time returned in ms.

`propose_enable_db_writes`

`{ reason }`

Ask permission to switch the bridge to write mode. You see a proposal card listing which databases gain write access and why. Flipping requires a bridge reinstall with `allowWrites:true`.

`propose_add_database`

`{ config_path, reason }`

Propose adding a database Pier didn't auto-discover (e.g. a custom connection file). Pier parses the config with every CMS parser and shows you the extracted credentials for review before connecting.

`propose_db_update`

`{ table, pk_column, pk_value, set: { ... }, reason, db_index? }`

`UPDATE` a single row. Pier pre-fetches the current row so you see a before/after diff. Column names are regex-validated. Max 10 columns per update. Executed with `LIMIT 1` under the hood.

`propose_db_insert`

`{ table, values: { ... }, reason, db_index? }`

`INSERT` one new row. Parameterized, no string interpolation.

`propose_db_delete`

`{ table, pk_column, pk_value, reason, db_index? }`

`DELETE` one row by primary key. Pier fetches and shows the row first so you see exactly what disappears if you accept.

Audits

`run_agent`

`{ agent, folder? }`

Run a specialized audit on up to 40 files, max 200 KB each, depth 3. Returns a list of issues plus auto-fix proposals where possible. Available agents: `seo`, `security`, `performance`, `accessibility`, `broken_links`, `meta_description`, `viewport`, `analytics`, `external_links`, `nofollow`, and `full_audit` (runs the lot).

Safe edits: proposals, diffs, undo

Every file write and every database write in Pier is a two-step process.

Pier User Guide v1.0

1. **The AI proposes.** It says "I want to change `functions.php` line 124 from X to Y, here's why." The proposal appears as a card in the chat with a full diff.
2. **You accept or reject.** Nothing touches your server until you click **Accept**. Rejecting doesn't re-prompt — you can respond with "actually, do it this other way" and the AI will draft a new proposal.

What a proposal card shows

- The file path (clickable — opens the full file in a modal).
- The **reason** the AI gave for the change. If the reason is vague or wrong, reject.
- Byte count before and after.
- A line-level diff. Removed lines in red, added lines in green.
- A **high-risk** amber badge for `.htaccess`, `wp-config.php`, `web.config`, `configuration.php`, `composer.json`, `package.json`. These files can take your site offline if broken, so we make them visually distinct.

What happens when you accept

1. Pier creates a timestamped backup of the current file (local, under `~/Library/Application Support/Pier/backups/`).
2. Pier uploads the new version over FTP/FTPS.
3. The server-side checksum is verified.
4. A 10-second **undo toast** appears in the bottom-right of the chat. Click **Undo** and Pier re-uploads the backup, instantly reverting.
5. After 10 s the toast fades — but the backup is still on disk, and you can restore it later from the file's version history (tool `list_backups`, or right-click the file in the sidebar → **History**).

Bulk accept

When the AI runs something like "replace every link to the old domain", it produces one proposal per file. You can accept them one-by-one, or use the **Accept all** button at the top of the chat to ship them in sequence. Each still gets its own backup and its own undo toast.

> A PROPOSAL IS NOT A PROMISE
>
> The AI occasionally proposes changes that look right but subtly aren't — especially in templating edge cases or localized strings. Read every diff. Pier can't replace your judgment, only remove the friction around exercising it.

Auto-audits

Pier User Guide v1.0

Audits are pre-packaged tours of your codebase. Each one scans up to 40 files (max 200 KB per file, depth 3) and returns issues plus — where safe — auto-fix proposals.

Agent	What it checks	Auto-fix?
`seo`	Title tags, meta descriptions, canonical links, Open Graph, structured data.	Suggestions only
`security`	Hardcoded secrets, `eval` usage, outdated hash functions, missing CSRF tokens, mixed-content assets, unescaped output.	Suggestions only
`performance`	Unminified assets, missing cache headers, oversized images, synchronous JS, render-blocking CSS.	Suggestions only
`accessibility`	Missing alt text, low-contrast colors, missing form labels, tab order issues, skipped heading levels.	Suggestions only
`broken_links`	Internal links to pages that don't exist, dead asset references.	Yes — proposes fixes
`meta_description`	Pages without a meta description, or descriptions longer than 160 chars.	Yes — drafts new ones
`viewport`	HTML pages missing `<meta name="viewport">`.	Yes
`analytics`	Detects analytics tags, checks for common misconfigurations (async missing, old UA property, duplicate GA).	Suggestions only
`external_links`	Outbound links missing `target="_blank" rel="noopener"`.	Yes
`nofollow`	Links that should be `rel="nofollow"` (affiliate, sponsored) but aren't.	Yes — with your approval
`full_audit`	Runs all of the above in sequence.	Mixed

To run one, say *"run a security audit on the active theme"* or *"audit this site for broken links"*. Pier will call `run_agent` and return the findings as a chat card; auto-fix proposals appear below, one per issue, each with its own Accept button.

Databases (MySQL)

Pier talks to your database through a small PHP script called the **bridge**. You install it to a randomised folder on your server, Pier talks to it over HTTPS, and (unless you opt in) it can only read.

Installing the bridge

Click **Databases → Install bridge** (or ask Pier in chat: *"set up the database bridge"*). Pier:

1. Generates a 128-bit random token.
2. Creates `/.pier-[token]/b.php` in the web root.
3. Uploads the bridge PHP script there.
4. Tests the connection by hitting `/.pier-[token]/b.php` with the token in both the `X-Pier-Auth` header and the request body.

If the test fails, Pier cleans up (the script is removed so it doesn't linger). Common reasons for failure: PHP disabled in that subdir, `.htaccess` rewriting the request, or the host blocking dotfolder URLs.

What the bridge allows

Mode	Allowed SQL	Blocked
Read-only (default)	`SELECT`, `SHOW`, `DESCRIBE`, `EXPLAIN`, `WITH`	Everything else.
Write (explicit opt-in)	Above + single-row `UPDATE` / `INSERT` / `DELETE` by primary key	`CREATE`, `ALTER`, `DROP`, range deletes, multi-row updates, multi-statement queries.

Enabling writes

1. Ask the AI to do something that requires a write — e.g. *"mark this product out of stock"*.
2. Pier calls `propose_enable_db_writes`. You see a card listing which databases will gain write access and why.
3. Click **Enable writes**. Pier reinstalls the bridge with `allowWrites:true`.
4. Pier proceeds with the row-level proposal (`propose_db_update`, `propose_db_insert`, `propose_db_delete`), which you Accept or Reject per row.

Safety rails, in one place

- **Row-level only.** The bridge enforces `LIMIT 1` on every mutation. There is no way to delete a range.
- **Primary-key required.** Every `UPDATE` or `DELETE` must name a `pk_column` / `pk_value`. No WHERE-by-other-column.
- **Column-name validation.** All column names checked against regex `[A-Za-z0-9_]+` before query.
- **Parameterized queries.** Values go through the mysqli prepared-statement pipeline; no string interpolation.

- **Pre-fetch-on-delete.** Before a delete, Pier `SELECT`s the row so you see what you're about to lose.
- **Dual auth.** Every bridge request carries the token in both the `X-Pier-Auth` header and the body. An attacker would need both to replay.
- **Optional IP allowlist.** You can restrict the bridge to specific client IPs (Pier's origin + your current address).
- **24-hour self-destruct.** After 24 hours with no authenticated request, the bridge deletes itself. Reinstall when you come back.
- **Row cap: 500.** Queries returning more than 500 rows are truncated with a flag, so the AI knows there's more.

What the bridge does *not* log

The bridge doesn't write any logs on your server. No request history, no query text, no error file. If something goes wrong, you only see it in Pier's in-app activity log — locally, on your Mac.

Backups & snapshots

Pier never edits your server without a backup. There are two kinds.

Per-file backups

Every time a file is edited, Pier saves a timestamped `.bak` of the previous version to `~/Library/Application Support/Pier/backups/[conn]/[path]/`. A `.checksums.json` file next to it stores SHA-256 checksums so restore can verify the file hasn't been tampered with locally.

To see history, right-click a file in the sidebar → **Show history**, or ask the AI: *"show me the version history for functions.php"*. Pier shows every version with its timestamp and size; click one to restore (which also creates a backup of the *current* version first, so restore is itself undoable).

Full-folder snapshots

A snapshot is a zip of the whole working folder. Use one before big refactors. To create: ask Pier *"snapshot this site as pre-refactor"* or use the **Backups → New snapshot** menu. Storage: `~/Library/Application Support/Pier/backups/[conn]/_snapshots/snap-[timestamp]-[name].zip`, with a matching `.meta.json` carrying file count, total size and SHA-256.

Restore is surgical: Pier extracts the zip to a temp folder, validates checksums (prevents zip-slip attacks), and then uploads files back over FTP one at a time, creating a fresh `.bak` of every overwritten file first.

Retention

By default Pier keeps the last 20 versions of each file, or 90 days of history — whichever hits first. Both are configurable in `settings.json` (Chapter 11). Retention runs automatically on every apply and before every delete.

> **WHERE YOUR BACKUPS LIVE**
>
> All backups are local. They take disk space but never network time. Typical mid-size WordPress site: ~20 MB of backups after a month of edits. Check `~/Library/Application Support/Pier/backups/` in Finder.

Settings & preferences

The settings file

Global settings live in `~/Library/Application Support/Pier/settings.json`. You can edit it by hand or change values through **Settings** in the app. Defaults:

```
{
  "backupRetention": {
    "mode": "count",          // "count" or "days"
    "keepLastN": 20,
    "keepDays": 90
  },
  "defaults": {
    "protocol": "ftp",        // "ftp" or "ftps"
    "port": 21,
    "readOnly": false
  },
  "autoBackupBeforeUpload": true,
  "maxSnapshotSize": 104857600  // 100 MB
}
```

Per-connection settings

Each connection carries its own model, BYO API key, and grouping label, stored encrypted in `connections.json`.

Field	Meaning
`model`	Which Claude model this connection uses. Defaults to `claude-opus-4-7`.
`apiKey`	Optional Anthropic key. Encrypted.
`openaiKey` / `openaiModel`	Optional OpenAI key for the BYO-OpenAI path.
`provider`	`anthropic`, `openai`, or `local-claude` (experimental).
`group`	A tag for organising connections in the sidebar.

Environment variables

Variable	What it does
`PIER_DATA_DIR`	Override the app data folder.
`PIER_READ_POOL_SIZE`	Number of parallel FTP reader clients. Default `0` (disabled). Set to `3` to speed up bulk reads on hosts that won't fail2ban you for parallel connections.
`PIER_LICENSE_API_BASE`	Point the license client at a different server. Mostly for our staging testing.

Variable	What it does
`ANTHROPIC_API_KEY`	Fallback key if a connection doesn't have one.
`FTP_MANAGER_PORT`	Change the localhost port Pier's backend binds. Default `3457`.

Keyboard shortcuts

Shortcut	Action
⌘ Enter	Send the current chat message.
⌘ K	Focus the chat input from anywhere.
⌘ S	Save the open file editor.
⌘ F	Search your chat history.
? or ⌘ /	Show the shortcuts help modal.
Esc	Close any open modal or cancel a pending action.

Plus the standard macOS suite from the menu bar:

Shortcut	Action
⌘ Q	Quit Pier.
⌘ M	Minimize window.
⌘ Z / ⌘ ⇧ Z	Undo / Redo in text fields.
⌘ X / C / V / A	Cut / Copy / Paste / Select All.

ONE MORE THING

There is intentionally no "apply immediately" shortcut. Every edit goes through the proposal → Accept flow. If we added a keyboard shortcut to skip the Accept, we'd have added a footgun.

Troubleshooting

Connection refused, "AUTH"

Username or password wrong, or the account is disabled. Reset the password in your hosting control panel and try again. After 3–5 wrong attempts most hosts temporarily IP-ban — see "BANNED" below.

"BANNED" / fail2ban

Shared hosts usually run fail2ban to block IPs after several failed logins. If Pier's circuit breaker opens (after 3 control-socket failures in 60 s), it holds you off for 180 s to prevent making it worse. Meanwhile the host itself may ban your IP for 15 minutes to an hour. Wait, or contact your host's support to unban your IP.

"TIMEOUT"

Firewall or wrong port. Try both passive and active modes from your router; confirm the port (21 for FTP, 990 some providers; FTPS usually upgrades on 21).

Edits apply but the site doesn't change

Three likely causes:

1. You're hitting a cache — server-side Varnish / Cloudflare / a caching plugin. Purge and retry.
2. Wrong file. Use `get_page_structure` to trace which template the request actually hits.
3. Read-only mode is on for this connection. Unchecking it re-enables writes.

Bridge install fails

- **PHP disabled in subdirectories.** Some hosts require `AddHandler php-cgi .php` in `.htaccess`. Check with your provider.
- **Dotfolders blocked.** A few hosts refuse to serve URLs beginning with a dot. In that case, rename the folder via the chat — ask Pier *"reinstall the bridge with a non-dot folder"*.
- **mod_security.** Overzealous WAFs sometimes 406 the POST. Ask your host to whitelist the bridge path.

License won't activate

- **"Signature does not match".** The pasted key got corrupted. Re-copy from the original email; Pier strips whitespace automatically.
- **"No more seats available".** Both seats are in use on other Macs. Deactivate one of them (Settings → License → Deactivate), or email `info@abn.company` if you've lost access to the original machine.
- **"Network error".** Check internet. The activate call goes to `pier.abn.company/api/license/activate`.

Chat says "license required" but I activated yesterday

Your 7-day offline grace has likely expired during a long offline stretch. Connect to the internet, open **Settings → License → Re-validate now**, and you're back.

"Working folder is too permissive"

Pier refuses to set `/` as a working folder. Pick a real subdirectory (`/public_html`, `/domains/example.com/public_html`, etc.) so the AI's scope is bounded.

Getting help

Bug reports, questions, feature requests — email `info@abn.company`. Replies arrive within one business day. If you want to watch the server logs while reproducing an issue, open Console.app and filter for `Pier/server`; logs live at `~/Library/Logs/Pier/server.log`.

Licenses & offline use

What a Pier license is

A Pier license is a signed payload that unlocks the AI features. The private key lives on our license server; the public key is embedded in Pier.app. When you paste your license, Pier verifies the signature offline (~0.3 ms) before it ever hits the network.

The 2-seat model

Every license covers two Macs. A seat is bound to a machine's **hardware UUID** (read from `IOPlatformUUID` via `ioreg`, with a hostname-SHA-256 fallback). Moving Pier between disks on the same Mac is free; cloning to a different Mac uses a second seat.

You can freely deactivate and re-activate as often as you like. If a Mac dies or you sell it, email us and we'll manually free the seat.

Offline behaviour

Days since last validate	What happens
0 – 6	Silent, everything works. Daily background re-validate attempts.
7 – 29	Yellow "please re-validate soon" banner in Settings. AI chat still works.
30 – 59	Red "Pier can't reach the license server" warning. AI chat still works.
60+	AI chat gated. File browsing, connection management, and existing backups still work. One re-validate re-opens everything.

Refunds

We offer a 14-day, no-questions-asked refund. Reply to the purchase email or mail `info@abn.company`. A refund automatically revokes your license.

Privacy & security

What lives only on your Mac

- FTP/FTPS credentials — AES-256-GCM encrypted in `connections.json`.
- Bring-your-own API keys (Anthropic, OpenAI) — same encryption.
- License payload + machine UUID binding — signed, stored locally.
- Chat history and proposals — JSON in `~/Library/Application Support/Pier/data/`.
- Every backup and snapshot.

What Pier sends over the network, and when

Destination	When	What
Your FTP/FTPS server	Every file operation	Standard FTP commands. Over FTPS the connection is TLS-encrypted end to end.
api.anthropic.com	Every AI chat turn	Your message, the relevant file content the model needs, and a summary of the conversation so far. Plus your API key in the header.
pier.abn.company	On first activate + every ~7 days	License ID, machine UUID, app version. No file content, no FTP credentials.
Your DB bridge URL	When you query the database	SQL (for `SELECT`s) or proposal parameters (for writes), plus the dual-auth token. Over HTTPS if your host supports it.

Secret redaction

Before any file content reaches the AI, Pier scrubs common secret patterns: PHP `define('KEY','VALUE')`, `.env`-style `KEY=VALUE`, assignments to variables whose names look sensitive, and regex matches for Stripe, AWS, and GitHub tokens. Truly sensitive paths (`wp-config.php`, `.env`, `.htpasswd`, `*.key`) are flagged and redacted before the read.

Telemetry

Pier v1.0 sends **no** telemetry, crash reports or usage analytics. A v1.1 opt-in error-reporting toggle is on the roadmap; it will default off and send only stack traces + app version, not file content or chat turns.

GDPR and EU law

Pier is built and sold by A Brand New Company (ABN B.V.), Amsterdam. Since the app does not transfer personal data off-device for its own purposes, there's no data processor relationship between us and you. When you use the AI, Anthropic acts as your sub-processor per their terms. Full privacy policy: `pier.abn.company/legal/privacy`.

FAQ

Why macOS only?

Pier is built around a signed + notarized macOS app with a native WKWebView wrapper. A Windows build is technically straightforward but a full support burden we're not taking on during v1. No timeline yet.

Can I use my own Anthropic key?

Yes. Connection → **Settings → API Key**. With BYO key, billing bypasses our shared quota and your usage counts against your Anthropic account.

Does the AI edit my live site without asking?

No. Every file write and every database write goes through a proposal you have to Accept. The only operations the AI executes without a proposal are *read* operations, `create_folder`, `rename_file`, and `set_working_folder`.

What about WordPress plugins / staging / Git workflows?

Pier edits the live filesystem you give it access to. If that's a staging server, Pier edits staging. If you want a git-based workflow, point Pier at your local working copy and use your usual `git` commands to ship changes — the AI plays nicely with local development directories too.

Can Pier delete my whole site?

In principle, yes — if you gave it write access, approved a snapshot deletion, and then asked it to. In practice, no individual chat turn can destroy a site. File deletes are one-at-a-time proposals; database deletes are single-row only. There is no "delete everything" primitive.

What happens if the license server is down?

Nothing visible for 7 days. After that, a yellow banner. After 60 days, AI chat is gated. File browsing always keeps working. The offline grace is generous on purpose so an infra outage on our side never locks a paying customer out in the middle of a job.

Can I install Pier on a machine I share with colleagues?

Yes, but each macOS user account is a separate seat. Pier's data is stored under each user's `~/Library/Application Support/Pier/`, so two users on one Mac means two activated seats.

Where's the admin dashboard?

The admin dashboard at `pier.abn.company/admin` is for ABN staff — revoking abused licenses, resending keys, fielding refund requests. You don't need it as a user.

Is my data used to train AI?

Anthropic's current terms for API usage exclude inputs from model training. Pier sends everything through the API (not a consumer product), so your code is not training data. If that ever changes, we'll flag it loudly in the blog.

I have a feature request.

Email `info@abn.company`. We read every request and the v1.1 / v1.2 / v2.0 roadmaps are driven largely by what customers ask for. Short, concrete use cases beat long feature specs — describe the job you're trying to get done.